CHAPTER 05 · PHASE 1: FOUNDATION

Nginx: Reverse Proxy & Web Gateway

Layer 7 request handling for the DevSecOps Lab edge — path-based routing, URL rewriting, WebSocket upgrades, and compression sitting behind HAProxy.



DevSecOps Lab Book — Zero to Hero

WHAT WE'LL COVER

Chapter Roadmap

Why Nginx Matters

Layer 4 vs Layer 7 — dividing work between HAProxy and Nginx

Install & Configure

Package install, modular config architecture, upstream pools

Path-Based Routing

Server blocks, location rules, URL rewriting per backend

Wire Up HAProxy

Point HAProxy at Nginx on :8080; test the full chain

Advanced Features

WebSockets, gzip compression, rate limiting, caching

Validate & Troubleshoot

Automated checks, monitoring, common failure modes

Layer 4 vs Layer 7



Layer 4 — Transport

HAProxy's Domain

- TCP/UDP connections
- IP-based routing
- Load balancing algorithms
- Failover and health checks
- Agnostic to application protocol (works for any protocol)

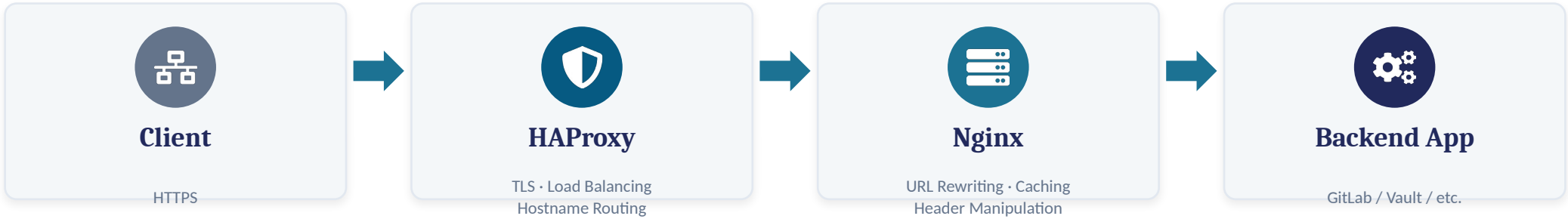


Layer 7 — Application

Nginx's Domain

- HTTP/HTTPS semantics
- URL path rewriting
- Request header inspection/modification
- Application-level caching & gzip compression
- WebSocket proxying
- Static file serving (with caching headers)

Request Flow — Client to Backend



Response flows back the same path — Nginx formats it, HAProxy delivers it over HTTPS

Analogy: HAProxy is the mail room of a building — it receives all incoming mail, checks the address, and routes it to the right department. Nginx is the department secretary — it reads the letter, files it properly, and formats the response nicely before sending it back.

Installing Nginx on sec-edge-01

1 **Connect to the edge host**

```
ssh sec-edge-01
```

2 **Install the package**

```
sudo apt update
sudo apt install -y nginx
```

3 **Verify the version**

```
nginx -v
→ nginx version: nginx/1.24.x
```

Port conflict: HAProxy and Nginx both default to 80/443. Stop and disable Nginx for now (sudo systemctl stop nginx && sudo systemctl disable nginx) — we'll reconfigure it to listen on 8080, internal-only, for HAProxy.

How Nginx Configs Are Organized

```
/etc/nginx/
├── nginx.conf          # Main config
├── conf.d/             # Additional configs
│   └── default.conf    # Default server block
├── sites-available/    # Potential site configs
├── sites-enabled/      # Symlinks to active sites
└── snippets/           # Reusable config blocks
```

```
sudo ln -sf /etc/nginx/sites-available/devsecops-lab \
    /etc/nginx/sites-enabled/devsecops-lab
sudo rm -f /etc/nginx/sites-enabled/default
```

Enable the site and remove the stock default config.

We build it in three layers:

1. **Main nginx.conf**
   Minimal — mostly includes
2. **Upstream definitions**
   One pool per backend service
3. **Server blocks**
   Virtual host per routing rule set

Backend Pools — One Upstream per Service

Upstream	Backend Target	Service
gitlab_backend	10.0.2.14:80	GitLab
vault_backend	10.0.2.13:8200	Vault
authentik_backend	10.0.2.11:9000	Authentik
lldap_backend	10.0.2.11:17170	LLDAP Web UI
harbor_backend	10.0.2.16:80	Harbor
sonarqube_backend	10.0.2.15:9000	SonarQube
defectdojo_backend	10.0.2.15:8000	DefectDojo
nexus_backend	10.0.2.16:8081	Nexus
kibana_backend	10.0.2.30:5601	Kibana
wazuh_backend	10.0.2.31:443	Wazuh
k8s_backend	10.0.2.20:6443	Kubernetes API



Health-aware pools

Every upstream sets `max_fails=3` and `fail_timeout=30s`, so Nginx stops sending traffic to a backend after 3 failures for 30 seconds.

Add a backup server line per pool for failover:

```
server 10.0.2.19:80
  backup;
```

The Main Server Block — Baseline Settings

```
server {
    listen 8080 default_server;
    server_name _;

    access_log /var/log/nginx/access.log combined;
    error_log  /var/log/nginx/error.log warn;
    server_tokens off;

    client_max_body_size 100M;

    proxy_connect_timeout 60s;
    proxy_send_timeout    60s;
    proxy_read_timeout    60s;
    ...
}
```



`server_tokens off`

Hides the Nginx version string from responses



`client_max_body_size 100M`

Prevents oversized uploads from being rejected too early



`proxy_*_timeout 60s`

Connect / send / read timeouts to upstream backends



`access_log combined`

Full request logging for later analysis and audits

The Location Block Pattern

Every backend gets the same three-step pattern — matched by URL prefix:

1 Match & strip the prefix

```
location ~ ^/gitlab(/|$) {
  rewrite ^/gitlab(/?.*)$ $1
break;
```

2 Proxy to the upstream

```
proxy_pass
http://gitlab_backend;
proxy_http_version 1.1;
```

3 Forward client context

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP
$remote_addr;
proxy_set_header X-Forwarded-
Proto $scheme;
```

Result: a client request to `https://10.0.2.12/gitlab/users` arrives at GitLab as `http://10.0.2.14/users` — the /gitlab prefix never reaches the backend.

Backends wired this way: GitLab · Vault · Authentik · LLDAP · Harbor · SonarQube · DefectDojo · Nexus · Kibana · Wazuh · Kubernetes API — plus a /healthz endpoint and a 404 catch-all.

Test the Config, Then Bring Nginx Up

```
$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
$ sudo systemctl start nginx
$ sudo systemctl enable nginx
$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Active: active (running) since ...
     Main PID: 5678 (nginx)
```



Always run nginx -t before every reload. A syntax error here means HAProxy's downstream target (port 8080) never comes up — catch it before it becomes an outage.

One Backend to Rule Them All: be_nginx

HAProxy no longer talks to application backends directly — every request now routes to Nginx on localhost:8080, which owns the application-level logic.

```
frontend fe_main
    bind 0.0.0.0:443 ssl crt /etc/haproxy/certs/devsecops.com.pem
    bind 0.0.0.0:80
    http-request redirect scheme https code 301 if !{ ssl_fc }
    default_backend be_nginx

backend be_nginx
    mode http
    option httpchk GET /healthz HTTP/1.1
    server nginx-01 127.0.0.1:8080 check
```

```
Configuration file is valid
```

Rollout steps

- Write the new config to haproxy.cfg.nginx
- Validate: sudo haproxy -c -f haproxy.cfg.nginx
- Back up the old config, swap in the new one
- sudo systemctl reload haproxy

Four Checks, End to End

1. **Health check**

```
curl http://127.0.0.1:8080/healthz
```

→ *OK*

2. **Direct Nginx request**

```
curl -v http://127.0.0.1:8080/gitlab/
```

→ *502 (expected — GitLab not deployed yet, but Nginx is routing)*

3. **Through HAProxy**

```
curl -k https://10.0.2.12/ -H "Host: gitlab..."
```

→ *502 Bad Gateway page (routing works, backend pending)*

4. **Check access logs**

```
sudo tail -10 /var/log/nginx/access.log
```

→ *Combined-format entries for every proxied request*

URL Rewriting & WebSocket Support

URL Rewriting

```
location ~ ^/gitlab(/|$) {
    rewrite ^/gitlab(/?.*)$ $1 break;
    proxy_pass http://gitlab_backend;
}
```

The `break` flag stops Nginx from re-evaluating the rewritten URL against other location blocks. Without it, the request may be processed again — and the prefix can resurface.

WebSocket Support

```
map $upstream_http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}

proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection $connection_upgrade;
proxy_buffering off;
```

Already applied to the GitLab and Authentik blocks — both rely on WebSocket connections for terminal sessions and real-time updates.

Compression & Rate Limiting

Gzip Compression

```
http {
    gzip on;
    gzip_vary on;
    gzip_proxied any;
    gzip_comp_level 6;
    gzip_types text/plain text/css
               application/json ...;
}
```

Reduces bandwidth for text-based responses (JSON, CSS, JS) before they leave the edge.

Rate Limiting

```
limit_req_zone $binary_remote_addr
  zone=general:10m rate=10r/s;
limit_req_zone $binary_remote_addr
  zone=login:10m rate=5r/m;

location /auth/flows/authenticate {
    limit_req zone=login burst=5 nodelay;
}
```

Caps login attempts per client IP to blunt brute-force attacks against Authentik.

HAProxy vs Nginx — Who Does What

Task	HAProxy	Nginx
TLS Termination	✅ Primary	⚠️ Secondary (could)
Host-based Routing	✅ Via ACL	✅ Via server_name
Path-based Routing	✅ Via ACL	✅ Via location
Load Balancing	✅ Excellent	✅ Good
Health Checks	✅ TCP/HTTP	✅ HTTP
WebSocket Support	✅ Automatic	✅ Via headers
Request Rewriting	❌ Limited	✅ Powerful
Failover	✅ Excellent	⚠️ Needs external tools

In this architecture

HAProxy:
TLS, hostname routing, failover, load balancing

Nginx:
URL rewriting, caching, compression, WebSocket protocol

5.12 VALIDATION SCRIPT

validate-chapter-05.sh — What It Checks

Nginx service active

```
systemctl is-active nginx
```

Config syntax valid

```
sudo nginx -t
```

Listening on :8080

```
netstat -tulnp | grep :8080
```

10+ upstreams defined

```
grep 'upstream ' upstreams.conf | wc -l
```

/healthz responds OK

```
curl http://$EDGE_IP:8080/healthz
```

HAProxy → Nginx routing

```
curl -k https://$EDGE_IP/ -H Host:...
```

Access logs populating

```
wc -l /var/log/nginx/access.log
```

Run from the HOST machine — script exits non-zero if any check fails, so it's safe to gate later chapters on it.

Common Problems & Fixes

Address already in use (:8080)

sudo lsof -i :8080 — kill the conflicting process or change Nginx's port

Nginx won't start

sudo systemctl status nginx or sudo journalctl -xeu nginx -n 20

502 Bad Gateway from HAProxy

Check Nginx is listening on 8080, the backend service is running, and Nginx can reach it directly with curl

Rewrite doesn't strip the prefix

Confirm the rewrite rule ends in break — without it, Nginx may re-process the URL

5.14 CHAPTER SUMMARY

What We Built

Nginx reverse proxy

Application-level request handling

Upstream definitions

Backend server pools

Path-based routing

Route different paths to different backends

URL rewriting

Remove path prefixes before sending to backends

WebSocket support

Real-time features in applications

Gzip + logging

Bandwidth reduction and full access-log visibility

KEY TAKEAWAY — SEPARATION OF CONCERNS

HAProxy: "Where does this traffic go?" **Nginx:** "How do I handle this traffic?"

This separation lets you change HAProxy without touching applications, add web-level features without changing routing, and scale each layer independently.